

# Техническое описание проекта «БренксЧат»

Веб-мессенджер: архитектура, технологии, база данных, хранимые данные и инфраструктура

## Краткая карточка проекта

| Параметр        | Значение                                                    |
|-----------------|-------------------------------------------------------------|
| Проект          | БренксЧат                                                   |
| Тип системы     | Веб-мессенджер с realtime-обменом сообщениями               |
| Основной стек   | React, TypeScript, Vite, Node.js, Express, Socket.IO, MySQL |
| Дата подготовки | 20 июня 2026 г.                                             |

## 1. Назначение проекта

«БренксЧат» представляет собой веб-мессенджер для обмена сообщениями в реальном времени. Система поддерживает личные чаты, группы, каналы, медиа-сообщения, голосовые сообщения, видеокружки, реакции, ответы, пересылку, закрепление сообщений, статусы прочтения, поиск пользователей, профили, push-уведомления, голосовые и видеозвонки. Также реализована административная часть для просмотра состояния системы и управления пользователями.

**Итог.** Проект является полноценным клиент-серверным приложением: клиент работает как SPA/PWA, сервер предоставляет REST API и Socket.IO-события, а постоянное состояние хранится в MySQL.

## 2. Общая архитектура

Архитектура построена по модели SPA + REST API + WebSocket + MySQL. Клиентская часть отвечает за интерфейс и локальные браузерные возможности, серверная часть — за авторизацию, валидацию прав, работу с сообщениями, realtime-события, почтовые коды, push-уведомления и подключение к базе данных.

- Клиент: одностраничное приложение React/TypeScript, собираемое Vite и стилизованное Tailwind CSS.
- Сервер: Node.js-приложение на Express и Socket.IO, запускаемое в production через systemd.
- База данных: MySQL-совместимое хранилище через библиотеку mysql2; в коде также сохранена совместимость с legacy JSON/app\_state.
- Realtime: Socket.IO используется для доставки сообщений, реакций, typing-состояний, presence, обновлений чатов и сигналинга звонков.
- Инфраструктура: Nginx обслуживает статические файлы, проксирует /api и /socket.io на Node.js, HTTPS настроен через Certbot.

## 3. Используемые технологии и библиотеки

Таблица 1 — Технологический стек проекта

| Слой           | Средство                            | Назначение                                                          |
|----------------|-------------------------------------|---------------------------------------------------------------------|
| Язык           | TypeScript                          | Основной язык клиентской и серверной разработки.                    |
| Клиент         | React 18                            | Компонентный пользовательский интерфейс мессенджера.                |
| Маршрутизация  | React Router                        | Маршруты входа, основного окна и админ-панели.                      |
| Сборка клиента | Vite                                | Dev-сервер, proxy /api и /socket.io, production-сборка client/dist. |
| Стили          | Tailwind CSS, PostCSS, Autoprefixer | Темы, адаптивность, модальные окна, сообщения и анимации.           |

| Слой                 | Средство                                                   | Назначение                                                              |
|----------------------|------------------------------------------------------------|-------------------------------------------------------------------------|
| Realtime-клиент      | socket.io-client                                           | Подключение к Socket.IO-серверу из браузера.                            |
| Криптография клиента | libsodium-wrappers, WebCrypto API                          | E2EE для текстов личных сообщений и резервные копии ключей.             |
| Сервер               | Node.js, Express                                           | REST API, middleware, отдача API и бизнес-логика.                       |
| Realtime-сервер      | Socket.IO                                                  | Сообщения, реакции, presence, typing и WebRTC-сигналинг.                |
| База данных          | MySQL + mysql2                                             | Постоянное хранение пользователей, чатов, сообщений и служебных данных. |
| Авторизация          | jsonwebtoken, cookie-сессии                                | Сессии пользователя и проверка доступа к API.                           |
| Пароли               | argon2                                                     | Хэширование паролей Argon2id.                                           |
| Почта                | nodemailer                                                 | Отправка кодов подтверждения, входа и восстановления пароля.            |
| Безопасность API     | helmet, cors, express-rate-limit                           | HTTP-заголовки безопасности, CORS и rate limit.                         |
| Push                 | web-push, Service Worker                                   | Уведомления о сообщениях и звонках.                                     |
| Desktop-артефакты    | Electron/electron-builder; отдельная папка Flutter desktop | Подготовка к desktop-версии и сборщикам Windows/macOS.                  |

## 4. Структура проекта

Таблица 2 — Основные каталоги и файлы

| Путь                         | Назначение                                                                                                       |
|------------------------------|------------------------------------------------------------------------------------------------------------------|
| client/src/pages             | Страницы LoginPage, MessengerPage, AdminPage.                                                                    |
| client/src/components        | Компоненты интерфейса: список чатов, окно чата, сообщение, строка ввода, профиль, просмотр фото, модальные окна. |
| client/src/contexts          | AuthContext, MessengerContext, CallContext, ThemeContext, ChatWallpaperContext.                                  |
| client/src/lib               | API-клиент, E2EE, поиск пользователей, хранение auth-данных, обои чата, утилиты отображения.                     |
| client/public                | PWA manifest, service worker, логотипы и иконки.                                                                 |
| server/src/index.ts          | Express-приложение, REST API, middleware, инициализация Socket.IO.                                               |
| server/src/socketHandlers.ts | Realtime-события сообщений, реакций, typing, прочтения, удаления, пересылки и звонков.                           |
| server/src/store.ts          | Бизнес-логика состояния: пользователи, чаты, сообщения, права, админ-обзор.                                      |
| server/src/persist.ts        | MySQL-схема, чтение и сохранение нормализованного состояния.                                                     |
| server/src/sessions.ts       | Активные сессии пользователей.                                                                                   |
| server/src/e2eeDevices.ts    | E2EE-устройства и резервные копии ключей.                                                                        |
| deploy                       | Nginx, systemd, backup timer/service и SSH-hardening.                                                            |
| electron                     | Electron-обертка для desktop-сборки.                                                                             |

| Путь    | Назначение                                     |
|---------|------------------------------------------------|
| desktop | Черновая/отдельная Flutter desktop-реализация. |

## 5. База данных

В проекте используется MySQL-совместимая база данных. Подключение выполняется через библиотеку mysql2 по строке DATABASE\_URL. Основная схема нормализована и создается автоматически при запуске сервера. В коде также присутствует таблица app\_state для совместимости со старым форматом хранения одного JSON-состояния, однако рабочая схема разделяет данные на отдельные таблицы.

**Количество таблиц.** По фактическому коду проекта создается 11 таблиц: app\_state, users, chats, chat\_participants, messages, user\_chat\_muted, user\_chat\_pinned, push\_subscriptions, auth\_sessions, user\_e2ee\_devices, user\_e2ee\_key\_backups.

**Таблица 3 — Таблицы базы данных и хранимые данные**

| Таблица           | Назначение             | Основные поля                                                                                                                                                                                            | Какие данные хранятся                                                                                                                                       |
|-------------------|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| app_state         | Legacy-совместимость   | id, data, updated_at                                                                                                                                                                                     | Единый JSON-снимок состояния старого формата. Используется как fallback при чтении старых данных.                                                           |
| users             | Пользователи           | id, username, password, email, email_verified, avatar_url, display_name, bio, phone, birth_date, is_admin, banned, privacy, updated_at                                                                   | Профили пользователей, хэши паролей, почта, аватар, отображаемое имя, описание, телефон, дата рождения, роль администратора, блокировка, privacy-настройки. |
| chats             | Чаты                   | id, type, name, avatar_url, last_message, unread, last_read_at, pinned_message_id, channel_owner_id, updated_at                                                                                          | Личные чаты, группы и каналы; аватар, последний preview, счетчики непрочитанных, прочтение, закрепленное сообщение, владелец канала.                        |
| chat_participants | Участники чатов        | chat_id, user_id, position                                                                                                                                                                               | Связь many-to-many между пользователями и чатами. Есть внешний ключ на chats(id) с ON DELETE CASCADE.                                                       |
| messages          | Сообщения              | id, chat_id, sender_id, text, encrypted_text, image_url, media_kind, media_data_url, media_file_name, media_mime_type, media_duration_ms, created_at, deleted, edited_at, reply_to_message_id, reactions | Текст/зашифрованный текст, медиа, файлы, голосовые, видеокружки, время, флаг удаления, редактирование, ответы и реакции.                                    |
| user_chat_muted   | Отключение уведомлений | user_id, chat_id                                                                                                                                                                                         | Персональная настройка пользователя: чат без звука.                                                                                                         |
| user_chat_pinned  | Закрепление чатов      | user_id, chat_id                                                                                                                                                                                         | Персональная настройка пользователя: чат закреплен наверху списка.                                                                                          |

| Таблица               | Назначение                  | Основные поля                                                  | Какие данные хранятся                                                                                                     |
|-----------------------|-----------------------------|----------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| push_subscriptions    | Push-уведомления            | user_id, endpoint, data                                        | Web Push-подписки браузеров: endpoint и JSON-ключи p256dh/auth.                                                           |
| auth_sessions         | Сессии входа                | id, user_id, created_at, expires_at                            | Активные авторизованные сессии, включая длительные сессии remember me.                                                    |
| user_e2ee_devices     | E2EE-устройства             | user_id, device_id, public_key, created_at, last_seen_at       | Публичные Curve25519-ключи устройств пользователя для шифрования личных сообщений.                                        |
| user_e2ee_key_backups | Резервные копии E2EE-ключей | user_id, version, salt, iv, ciphertext, iterations, updated_at | Зашифрованная резервная копия приватного ключа устройства; шифруется паролем пользователя через WebCrypto/PBKDF2/AES-GCM. |

## 5.1. Связи и индексы

- chat\_participants.chat\_id связан с chats.id внешним ключом; при удалении чата участники удаляются каскадно.
- messages.chat\_id связан с chats.id внешним ключом; при удалении чата сообщения удаляются каскадно.
- messages имеет индекс idx\_messages\_chat\_created по chat\_id и created\_at для быстрой загрузки истории чата.
- chat\_participants имеет индекс idx\_chat\_participants\_user для поиска чатов пользователя.
- auth\_sessions имеет индексы по user\_id и expires\_at для очистки просроченных сессий.
- user\_e2ee\_devices имеет составной первичный ключ user\_id + device\_id.
- push\_subscriptions имеет составной первичный ключ user\_id + endpoint(191).

## 5.2. Особенности хранения медиа

Медиафайлы хранятся в таблице messages внутри полей media\_\* и image\_url. По структуре проекта данные медиа передаются как data URL и сохраняются в LONGTEXT. Такой вариант удобен для учебного проекта и простого деплоя, но при росте объема данных логично вынести файлы в объектное хранилище или отдельное файловое хранилище, оставив в MySQL только ссылки и метаданные.

## 6. Реализованный функционал

Таблица 4 — Функциональные блоки

| Блок     | Что реализовано                                                                                               |
|----------|---------------------------------------------------------------------------------------------------------------|
| Аккаунты | Регистрация, вход, подтверждение почты, сброс пароля, remember me, выход, профиль, аватар, privacy-настройки. |
| Чаты     | Личные чаты, избранное, группы, каналы, добавление участников, закрепление чатов, mute-режим.                 |

| Блок              | Что реализовано                                                                                        |
|-------------------|--------------------------------------------------------------------------------------------------------|
| Сообщения         | Отправка, редактирование, удаление, ответы, реакции, пересылка, закрепление, поиск, прочтение, typing. |
| Медиа             | Фотографии, файлы, голосовые сообщения, видеокружки, просмотр изображений, вставка/загрузка медиа.     |
| Звонки            | WebRTC-аудио/видео звонки, сигналинг через Socket.IO, ICE-серверы, push-уведомления о входящем звонке. |
| Безопасность      | Argon2id, cookie/JWT-сессии, rate limit, Helmet, CORS, email-коды, E2EE для текстов личных сообщений.  |
| Администрирование | Админ-панель, обзор системы, блокировка пользователей, ссылка на базу данных/phpMyAdmin.               |
| PWA               | manifest.json, service worker, push-уведомления, standalone-режим на телефоне.                         |

## 7. API и realtime-взаимодействие

Серверная часть предоставляет REST API для авторизации, пользователей, чатов, сообщений, E2EE-устройств, звонков, админ-панели и профиля. Для событий, которые должны приходить мгновенно, используется Socket.IO.

Таблица 5 — Основные REST API-группы

| Группа          | Маршруты                                                                                                               |
|-----------------|------------------------------------------------------------------------------------------------------------------------|
| Авторизация     | /api/register, /api/login, /api/login/confirm, /api/logout, /api/password-reset/*                                      |
| Профиль         | /api/me, /api/me/email/request, /api/me/email/confirm                                                                  |
| Пользователи    | /api/users/directory, /api/users/contacts, /api/users/search, /api/users/:userId                                       |
| Чаты            | /api/chats, /api/chats/:chatId/messages, /api/chats/direct, /api/chats/group, /api/chats/channel, /api/chats/saved     |
| Настройки чатов | /api/chats/:chatId/mute, /pin-top, /pin-message, /clear, /members, PATCH /api/chats/:chatId, DELETE /api/chats/:chatId |
| E2EE            | /api/e2ee/devices, /api/e2ee/key-backup, /api/chats/:chatId/e2ee-devices                                               |
| Звонки          | /api/calls/ice-servers                                                                                                 |
| Админ-панель    | /api/admin/overview, /api/admin/database, /api/admin/users/:userId/block                                               |

Таблица 6 — Основные Socket.IO-события

| Событие         | Назначение                                |
|-----------------|-------------------------------------------|
| join_chat       | Вход клиента в комнату чата.              |
| send_message    | Отправка сообщения и рассылка участникам. |
| edit_message    | Редактирование сообщения.                 |
| delete_message  | Удаление сообщения.                       |
| toggle_reaction | Добавление или снятие реакции.            |

| Событие                                                              | Назначение                                                               |
|----------------------------------------------------------------------|--------------------------------------------------------------------------|
| forward_messages                                                     | Пересылка сообщений в другой чат.                                        |
| mark_read                                                            | Отметка сообщений как прочитанных.                                       |
| typing                                                               | Индикатор набора текста.                                                 |
| presence                                                             | Список пользователей онлайн с учетом privacy-настроек.                   |
| call_signal                                                          | WebRTC-сигналинг: offer, answer, candidate, end и другие сигналы звонка. |
| chat_updated/message_edited/<br>message_deleted/<br>messages_cleared | События синхронизации состояния интерфейса.                              |

## 8. Безопасность

- Пароли пользователей хранятся не в открытом виде, а как Argon2id-хэши.
- Авторизация работает через серверные сессии, связанные с cookie/JWT.
- Для входа, регистрации и сброса пароля используются email-коды.
- На авторизационные маршруты и API установлен express-rate-limit.
- Helmet добавляет защитные HTTP-заголовки; Nginx дополнительно задает HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy и CSP.
- Тексты личных сообщений могут шифроваться на клиенте: envelope хранится в messages.encrypted\_text, а ключи устройств — в user\_e2ee\_devices.
- Резервная копия E2EE-ключа хранится зашифрованной и не содержит открытый приватный ключ.
- Production-сервис systemd работает от отдельного пользователя brenkschat и с hardening-настройками.

## 9. Развертывание и инфраструктура

Таблица 7 — Инфраструктурные элементы

| Элемент         | Описание                                                                                                                 |
|-----------------|--------------------------------------------------------------------------------------------------------------------------|
| Домен и HTTPS   | Nginx обслуживает silentx.ru и www.silentx.ru; HTTP перенаправляется на HTTPS; сертификат указан как managed by Certbot. |
| Статика         | client/dist отдается напрямую Nginx; /assets кешируются на 1 год как immutable.                                          |
| API             | /api проксируется на Node.js-сервер http://127.0.0.1:3002.                                                               |
| WebSocket       | /socket.io проксируется на тот же backend с Upgrade-заголовками и долгими timeout.                                       |
| База данных     | MySQL используется как основное persistent-хранилище; phpMyAdmin доступен через защищенный basic auth путь.              |
| Сервис          | systemd-unit brenkschat.service запускает /var/www/brenkschat/server/dist/index.js.                                      |
| Резервные копии | deploy/backup содержит Node.js-скрипт mysqldump + архив исходников + checksums + retention 7 дней.                       |

## 10. Сборка, тестирование и отладка

- Корневая команда `npm run build` последовательно собирает client и server.
- Клиентская сборка: `tsc -b && vite build`.
- Серверная сборка: `tsc`.
- Серверные тесты запускаются командой `npm test -w server`.
- В проекте есть тесты `password.test.ts`, `sessions.test.ts`, `e2eeDevices.test.ts`, `store.security.test.ts`.
- Тестами покрыты Argon2id-пароли, legacy-миграция паролей, сессии remember me, E2EE-устройства и резервные копии ключей.

## 11. Итоговое описание данных

Основные данные проекта делятся на пользовательские данные, коммуникационные данные, служебные настройки и защитные данные. Пользовательские данные включают профиль, аватар, почту, телефон, дату рождения и privacy-настройки. Коммуникационные данные включают чаты, участников, сообщения, медиа, реакции, ответы, статусы прочтения и закрепления. Служебные данные включают сессии, push-подписки, mute/pin-настройки и legacy-состояние. Защитные данные включают Argon2id-хэши паролей, E2EE-устройства и зашифрованные резервные копии ключей.

## 12. Заключение

Проект «БренксЧат» является полнофункциональным веб-мессенджером с современной клиент-серверной архитектурой. В нем используются React и TypeScript для интерфейса, Node.js/Express и Socket.IO для серверной части и обмена в реальном времени, MySQL для постоянного хранения данных, а также дополнительные механизмы безопасности, push-уведомлений, E2EE, WebRTC-звонков и production-деплойа. База данных состоит из 11 таблиц, которые покрывают пользователей, чаты, сообщения, сессии, уведомления и ключи шифрования. Такая структура позволяет развивать проект дальше: выделить файловое хранилище для медиа, расширить роли пользователей, улучшить аудит безопасности и масштабировать realtime-сервер.